通用原型设计工作指南

> 适用范围：适用于产品原型、页面草图、交互结构、流程页、运营后台、终端工具、平台控制台等大多数原型设计任务。
> 目标：在开始画页面之前，先把页面真正要解决的问题、它依赖的数据、它和其他页面的关系，以及用户会经历的状态变化讲清楚。

1. 原型设计到底在设计什么

很多团队一提原型，最先想到的是布局、按钮和模块摆放。但原型真正困难的地方，通常并不在视觉草图，而在更前面的判断：这页到底服务什么场景，围绕什么对象展开，用户为什么会来到这里，做完之后又会回到哪里。

如果这些问题没有先说清楚，页面画得越完整，后面返工往往越大。因为返工时改掉的不是一个按钮名，而是整页的中心、数据来源和前后链路。

所以原型设计首先是在设计三件事：

1. 页面职责。
2. 页面关系。
3. 页面状态。

视觉表达当然重要，但它通常发生在这些判断之后，而不是之前。

2. 先从场景开始，不要先从页面长相开始

一个页面是否合理，首先取决于它出现在哪个场景里。

同样是“查看信息”，有的页面只是日常浏览，有的页面其实承接高风险动作前的确认；同样是“修改配置”，有的页面只改一项，有的页面其实会影响整批对象；同样是“进入下一步”，有的只是普通跳转，有的却意味着任务正式开始执行。

因此在动手写页面之前，最好先用完整的话把场景说出来：

1. 谁会来到这页。
2. 他是在什么情况下来到这页。
3. 他来到这页最想先确认什么。
4. 他在这页做完以后，下一步通常是什么。

当这几句话能说清楚时，页面边界往往也会更自然。反过来，如果场景本身含糊，页面就很容易越画越像功能堆叠，而不是一条能走通的工作路径。

3. 先分清对象、动作和范围

原型里最常见的错误之一，是把“对象本身”“这次动作”“这次动作影响的范围”写到一起，最后用户看到的是一堆字段，但不知道哪部分是长期事实，哪部分只是当前这一轮临时变化。

比较好的做法，是先问清三个问题：

1. 这页围绕的主对象是什么。
2. 这页承接的主要动作是什么。
3. 这次动作影响的是整个对象，还是只影响当前选中的范围。

这一步看起来简单，实际上会决定整页怎么展开。

例如，某个批量任务里跳过两台设备，改变的只是这次动作范围，不代表整个对象集合真的被改写了；某次单机修正动作，只影响当前这一台设备，不应该被写成教室级页面；某页里用户只是设置当前任务参数，也不应该轻易被包装成一种新的长期业务对象。

还有一类常见误差，是把系统动作视角和持久化存储视角混在一起。比如页面上的“设置”可能只是用户这次输入的任务参数，不等于系统已经保存的“配置”。原型里最好先把视角词、动作词和存储词拆开，再决定字段和按钮怎么命名。

原型一旦在这里判断错了，后面字段定义、按钮动作和状态说明都会跟着偏。

4. 页面关系要先于页内布局

一页原型能不能成立，不只取决于页内写得清不清楚，还取决于它放在整个页面链路里是否合理。

在开始细化页内结构前，至少应先回答：

1. 用户从哪一页进入。
2. 完成后回哪一页。
3. 结果会回写到什么地方。
4. 这页是独立页、抽屉、弹层，还是当前页的一个状态。

如果这里没有先理顺，后面就很容易出现几种问题：同一件事被两页重复承接；本应在当前上下文里完成的动作被硬拆出去；流程被截成几段，每一段单看都像成立，但串起来却断了。

如果一个任务同时牵涉多台设备、多个操作者或多个终端状态，还需要再补两句判断：当前这页属于谁的屏幕，其他设备此时又分别停留在哪一页。否则页面看起来很完整，实际却把多个视角混成了一页。

5. 数据要先拆层，再写字段

很多原型之所以越写越乱，不是因为字段不够多，而是因为不同性质的数据被混在了一起。

比较常见的几层数据是：

1. 当前实时状态，例如当前在线状态、当前识别信息、当前网络或连通情况。
2. 本次任务数据，例如本次选择结果、本次参数、本次进度、本次失败项。
3. 历史持久化数据，例如上次写入记录、已保存配置、最近一次成功结果。
4. 外部系统返回数据，例如服务端登记结果、共享状态、平台汇总结果。

这些层次看似都能出现在一页上，但含义完全不同。实时状态回答的是“现在怎样”，任务数据回答的是“这次打算怎样”，持久化数据回答的是“系统长期记住了什么”，外部系统数据回答的是“别的地方知道什么”。

字段只有先放回正确的数据层，后面用户看到它时才能判断它到底能证明什么，不能证明什么。

实际写字段时，至少还要额外检查一组高频三层关系：当前值、已保存配置值、这次准备写入值。它们可以并列对照，但不能混成一个模糊字段；否则用户既看不清当前状态，也判断不了这次动作到底会改什么。

6. 字段定义不要只写名字，要写它为什么存在

原型里真正需要定义的，不只是字段名，而是字段在页面里的职责。

一个关键字段，至少应回答下面几个问题：

1. 它表示什么。
2. 它从哪里来。
3. 它什么时候应该出现。
4. 它为空时如何表达。
5. 用户能拿它做什么判断。
6. 用户不能仅靠它下什么结论。

最后两点特别重要。很多页面的问题，不是字段缺失，而是字段被误用了。某个字段本来只是辅助核对，却被写成唯一依据；某个字段本来只是当前状态，却被误当成长期配置；某个字段本来只是历史记录，却被拿来代表当前真实结果。

所以字段定义不是在补注释，而是在限制误解。

7. 流程页不能只有开始和结束

流程页最容易犯的错误，是只画入口和结果，中间最关键的骨架反而省掉了。

一条完整的流程，通常至少应把下面几类环节讲清楚：

1. 当前要处理什么对象，或当前选中了哪些对象。
2. 是否需要先筛选、勾选、排序或确认范围。
3. 是否需要选择来源对象、目标对象或当前要使用的版本。
4. 如果中途会进入别的环境，回来以后如何接续。
5. 执行前是否有最终确认和可视化预览。
6. 执行时如何表达阶段、进度、失败和重试。
7. 完成后用户下一步该去哪里。

不同产品的流程长度会不同，但只要它承接的是高风险动作、长任务或批量任务，这几类环节通常都不能被一句“然后系统开始处理”代替。

还有一条经常被漏掉的硬规则：只要流程正文里出现关键按钮、链接、入口或“查看详情/查看结果”这类动作，就必须继续交代它由什么承接形式接住。是进入独立页、打开弹层、展开明细、切换当前页状态，还是立即执行并在原地反馈，都要写清。按钮如果出现了，后文却没有承接，等于流程没有写完。

8. 状态不是补充项，而是页面的一部分

原型不能只写主态。用户真正容易卡住的地方，往往恰恰是那些默认不会被优先想到的状态。

至少要主动检查这几类状态：

1. 默认态：用户刚来到这页时，页面给他的起点是什么。
2. 空态：当前没有数据，是因为还没开始、前提不满足，还是确实为空。
3. 加载态或执行中：系统正在做什么，用户要等什么，还能不能改。
4. 成功态：任务完成后，用户下一步最常见的动作是什么。
5. 失败态：失败发生在哪里，用户接下来应该重试、修改还是返回。
6. 部分成功：哪些已经完成，哪些还没完成，页面现在最应该把注意力放在哪里。

这些状态不是为了把页面写得更“完整”，而是为了让用户在非理想情况下仍然知道系统当前发生了什么，以及自己下一步应该做什么。

部分成功尤其不能一句带过。它通常需要同时保留已完成对象、未完成对象，以及保持现状或未纳入对象。只要这几类对象会影响用户下一步判断，就不应被压缩成一个总数。

9. 什么时候该拆页，什么时候留在当前页

拆页不是越多越清楚，也不是越少越高级。关键在于当前动作是否已经超出了原页面上下文能自然承接的范围。

比较适合拆成独立流程页的情况通常包括：动作风险高；存在明显前后阶段；执行时间长；结果会影响多个对象或多个页面。

比较适合留在当前页里的情况通常包括：只是当前对象上下文里的补充动作；做完以后用户还要回到当前页继续工作；它本质上只是确认、查看或小范围调整。

弹层或抽屉更适合承接二次确认、局部补充输入和单对象核对。它们的优点是打断范围小，但前提是动作本身也确实足够局部。如果本来就是一个完整流程，就不该勉强塞进弹层。

10. 技术细节什么时候应该进入原型

一个技术细节是否应该进入原型，可以用一个简单标准判断：它会不会改变用户的选择、路径、结果理解或风险判断。

如果会，它就不只是技术实现问题，而应该进入页面表达；如果不会，暂时只影响内部实现，就不必过早写成页面字段或用户动作。

这条判断很重要，因为很多原型一方面会把不该出现的内部实现写进页面，另一方面又会漏掉那些其实已经影响用户选择的技术限制。结果就是：页面里多了很多用户不关心的细节，真正关键的选择却没被显式呈现出来。

11. 评审原型时，应该怎么问

评审原型时，可以用枚举帮助梳理，但每个问题都应该带着具体判断去问，而不是只列一个标题。

1. 这页的主对象是什么。
   说明：先确认页面到底围绕谁展开，否则后面关于字段、动作和状态的讨论很容易失焦。
2. 用户从哪来，到哪去。
   说明：如果前后链路说不清，这页即使内容完整，也可能只是孤立页面。
3. 当前页改的是对象本身，还是这次动作范围。
   说明：这一点会直接影响页面是不是在偷偷改写长期事实。

4. 关键字段从哪里来。
   说明：字段来源不清楚，后面就很难判断它到底是当前值、历史值，还是任务中的临时值。
5. 页面里有没有把未定机制写成既定事实。
   说明：这是原型最容易越界的地方，尤其在复杂流程和机制设计里更常见。
6. 异常、执行中、失败和部分成功有没有被补齐。
   说明：如果这些状态没有承接好，用户最容易在实际使用时失去方向。
7. 同一对象有没有在标题、正文、按钮和结果态里被写成多套叫法。
   说明：如果术语本身包含页面视角词、系统动作词和持久存储词的差异，应先定义规则，再逐处替换，不要靠半替换让页面看起来像是“已经统一”。
8. 当前技术限制里，有没有哪些已经影响用户选择，但页面还没表达出来。
   说明：这类问题如果漏掉，原型评审时往往看不出来，开发或联调阶段才会集中爆发。